

# 09 · Paralleles Rechnen (Multicore) & Finale

Mikrocomputertechnik 3 · Von SMP, MESI-Kohärenz und Hardware-Locks zum fertigen SoC

Prof. Dr.-Ing. Daniel Klünder

**Folie**

1 / 51

## Willkommen zum Finale!

- Neunte und letzte Vorlesung in Mikrocomputertechnik 3
- Weg: Speicherzelle → Caches → DMA → Busse → OoO → Edge AI
- Heute: Multi-Core — Thread-Level Parallelism auf dem SoC
- Praxis: Theorie zu MESI/Atomics, danach Wrap-Up und Klausurvorbereitung

## Notizen

Letzter Baustein: zweiter (und weiterer) Prozessorkern — und wie Software damit sicher umgeht.

**Folie**

2 / 51

## Agenda und Lernziele

1. Cache-Kohärenz und MESI (Bus Snooping)
2. Race Conditions und Hardware-Atomics (LR/SC, AMO)
3. Amdahls Gesetz und Semester-Rückblick / Klausurfokus

Lernziele:

- Kohärenzproblem und MESI-Zustände erklären
- Race Conditions auf Assembler-Ebene begründen
- Software-Locks vs. Hardware-Atomics unterscheiden
- Speedup-Grenzen mit Amdahl abschätzen
- Systemwissen der Blöcke 1–8 für die Klausur strukturieren

## Notizen

Zwei Probleme: Hardware-Kohärenz der Caches; Software-Atomizität bei Shared-Updates.

## Anknüpfung: Edge-AI-Labor

Kurz-Recap aus Block 8 (Vertiefung im Übungsblatt):

- TFLM-Inferenz auf dem RISC-V-Kern
- Matrix-Last blockiert die General-Purpose-CPU lange
- Ohne RVV/NPU: Memory Wall spürbar
- DLP vs. ILP: Spezialist vs. Alleskönner

### Notizen

Heute wieder Alleskönner-Kerne: OS und gemischte Tasks brauchen TLP, nicht nur Matrizen.

## Erinnerung: Taktraten-Mauer

- Warum Multicore? Power Wall (Block 7)
- Höherer Takt → stark steigende Verlustleistung und Kühlbedarf
- Besonders kritisch bei Mobile/Embedded
- Ausweg: mehr Kerne statt immer höherer GHz

### Notizen

Plan B der Industrie nach dem GHz-Wettrennen: Parallelität auf Thread-Ebene.

## Multi-Core: Thread-Level Parallelism

- Zwei (oder mehr) Kerne auf einem Chip statt „doppelt so schneller“ Einzelkern
- TLP: unabhängige Threads/Tasks parallel
- Beispiel: UI/Media auf Kern 0, Hintergrundarbeit auf Kern 1
- Oft bessere Energieeffizienz als ein extrem hoch getakteter Kern

### Notizen

Zweiter Arbeiter statt einen zu überhitzen — aber Kommunikation wird zum Problem.

## Symmetric Multiprocessing (SMP)

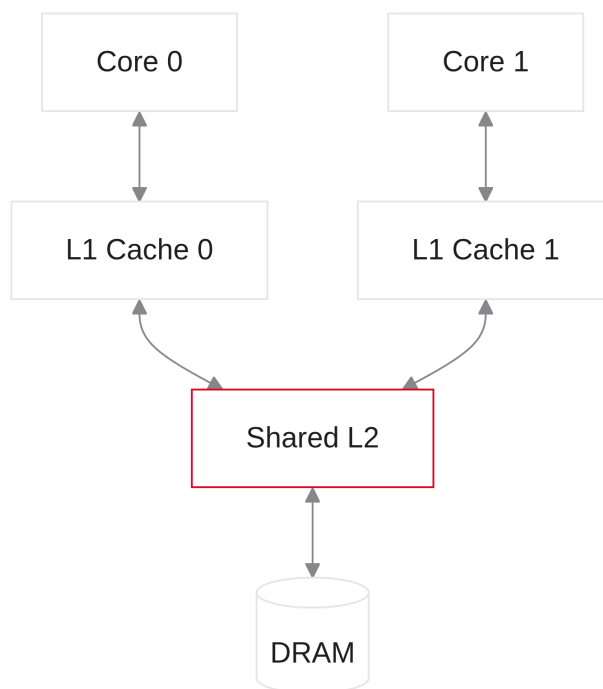
- SMP: alle Kerne gleichartig (symmetrisch)
- Gemeinsamer Zugriff auf RAM und Peripherie
- OS (Linux, FreeRTOS, ...) kann Tasks migrieren
- big.LITTLE u. Ä. hier ausgeblendet — Fokus SMP

### Notizen

Klassisches PC-/Phone-Modell: gleichberechtigte Kerne, Scheduler verteilt.

## Architektur eines SMP-SoC

- Pro Kern: privater L1 (I und D)
- Geteilt: oft L2 und DRAM (Shared Memory)



### Notizen

Private L1 für Tempo — darunter treffen sich die Kerne. Dort lauert die Kohärenzfrage.

### Die Leitfrage des Tages

- Write-Back (Block 2): Writes oft nur im privaten L1 — RAM kann veraltet sein
- Core 0 schreibt `score = 10` in seinen L1
- Core 1 will `score` lesen
- Woher weiß Core 1, dass die Wahrheit im L1 von Core 0 liegt — nicht im alten RAM?

### Notizen

Eine CPU: Write-Back genial. Zwei CPUs: ohne Kohärenz zerbricht die Semantik.

### Kohärenzverlust (Stale Data)

- Private Write-Back-Caches können unterschiedliche Sichten auf dieselbe Adresse erzeugen
- Kohärenz (pro Adresse): alle Kerne sehen eine einheitliche Reihenfolge der Writes
- Ohne Hardware-Kohärenz: OS-Flags, Queues, Sync unbrauchbar

### Notizen

Kohärenz  $\neq$  volle Speicher-Konsistenz für Mehrwort-Operationen — dafür später Atomics/Locks.

### Warum Software-Flush scheitert

- Bei DMA: gezielte Cache-Maintenance vor/nach seltenen Transfers
- Bei Multicore: tausende Shared-Variablen pro ms (Queues, Locks, ...)
- Manuelles Spülen nach jedem Write zerstört den Multicore-Gewinn
- Braucht unsichtbare, schnelle Hardware-Kohärenz

### Notizen

Write-Through überall wäre korrekt, aber Bus und Energie würden kollabieren.

## Bus Snooping

- Klassisch: Cache-Controller lauschen am gemeinsamen Bus/Interconnect
- Fremder Zugriff auf lokal Modified → eingreifen (Daten liefern / Invalidate)
- Große Systeme oft directory-basiert statt reinem Broadcast-Snoop

### Notizen

Idee bleibt: Caches koordinieren die „Wahrheit“ einer Adresse — Implementierung variiert.

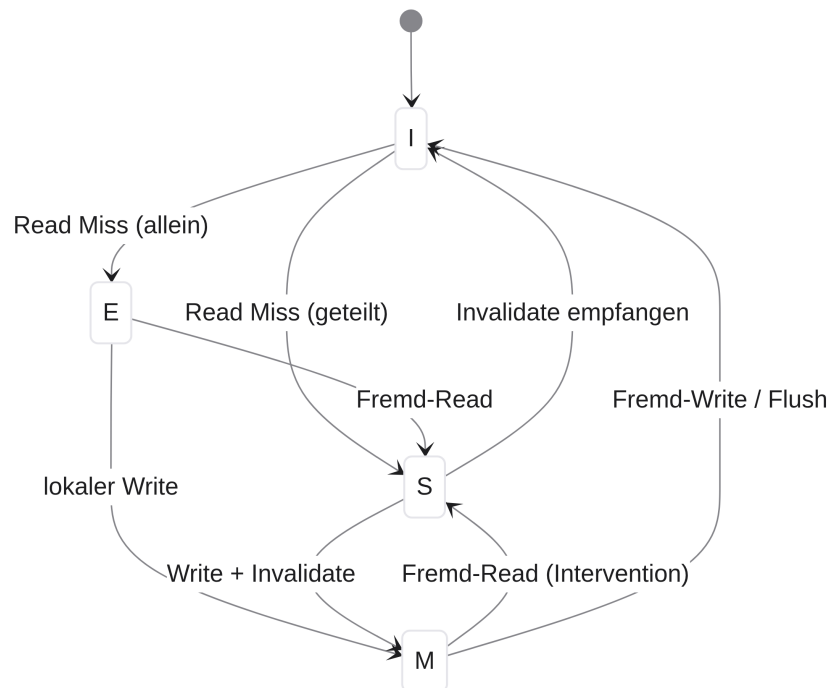
## MESI-Protokoll

- State Machine pro Cache-Line (typisch 64 Byte)
- Zustände: Modified (M), Exclusive (E), Shared (S), Invalid (I)
- Basis vieler kohärenter Multicore-Designs (Varianten: MOESI, ...)

### Notizen

Konzept der vier Buchstaben zählt — nicht jede Transitionstabelle auswendig.

## MESI: Zustandsübersicht



### Notizen

Die Pfeile sind die klausurrelevanten Übergänge. Merken Sie sich besonders: Write auf Shared braucht Invalidate; Read-Miss auf Modified liefert der Owner.

### Folie

14 / 51

### Zustand M (Modified)

- Line lokal geändert — RAM veraltet
- Exklusiv: keine andere gültige Kopie
- Klassischer Write-Back-„Boss“-Zustand

### Notizen

Monopol auf die Wahrheit dieser Adresse.

**Zustand E (Exclusive)**

- Sauber: Cache = RAM
- Exklusiv: niemand sonst hat die Line
- Write → oft still zu M, ohne Broadcast

**Notizen**

Sauber und allein — Schreiben ohne Netzwerk-Meeting.

**Zustand S (Shared)**

- Sauber, aber mehrere Kerne haben Kopien
- Gut für paralleles Lesen
- Vor lokalem Write: andere Kopien invalidieren

**Notizen**

Read-only-Sharing effizient — Write braucht Kohärenzaktion.

**Zustand I (Invalid)**

- Nach Reset oder Invalidate
- Inhalt unbrauchbar → Miss beim Zugriff
- Daten müssen neu geholt werden

**Notizen**

Leerer Eimer — nicht verwenden.

**MESI: Read-Miss**

Core 0 liest  $x$ , lokal I:

- Request auf dem Interconnect
- Niemand hat  $x$  → Memory liefert, Core 0 oft → E
- Anderer Kern hat  $x$  sauber (E/S) → beide → S
- Anderer Kern hat  $x$  Modified → der Owner liefert (Write-Back/Intervention), Zustände typisch → S

**Notizen**

Snooping/Directory entscheidet: allein, geteilt oder Owner-Supply bei Modified.

**MESI: Write auf Shared**

- Core 0 und 1:  $x$  in S
- Core 0:  $x = 5$
- Vor Write: Invalidate-Broadcast
- Dann lokal schreiben, Zustand → M

**Notizen**

Geteilte Ressource nie heimlich ändern.

**Reaktion: Invalidate**

- Core 1 sieht Invalidate für  $x$  → lokal I
- Späteres Lesen → Miss → frische Kopie (Wert 5) von Core 0 / Memory-Pfad

**Notizen**

Konsistenz: erzwungenes Verwerfen, dann Nachladen der Wahrheit.

## MESI-Beispiel: Zustandsfolge

Zwei Kerne, Variable  $x$ , Start beide Invalid:

| Schritt | Aktion                | Core 0 | Core 1 |
|---------|-----------------------|--------|--------|
| 1       | Core 0 liest $x$      | E      | I      |
| 2       | Core 1 liest $x$      | S      | S      |
| 3       | Core 0 schreibt $x=5$ | M      | I      |
| 4       | Core 1 liest $x$      | S      | S      |

- Schritt 3: Invalidate-Broadcast, dann lokal M
- Schritt 4: Owner-Supply (Intervention) oder Memory-Pfad → beide S

### Notizen

Typische Klausurfrage: Tabelle ausfüllen. Üben Sie die Sequenz Read-Miss → Shared → Write → Invalidate → erneutes Lesen.

## Overhead von MESI

- Preis: Kohärenz-Traffic (Invalidate, Snoop)
- Viele Kerne + häufige Shared-Writes → Interconnect belastet
- Große Server: Ringe/Meshes statt einfachem Broadcast-Bus

### Notizen

Skalierung: Verwaltungslast kann Datenbandbreite fressen.

## False Sharing

- Core 0: `temp1`, Core 1: `temp2` — logisch unabhängig
- Liegen in derselben Cache-Line → Write invalidiert den Partner
- Ping-Pong trotz verschiedener Variablen
- Abhilfe: Alignment / Padding auf getrennte Lines

## Notizen

Hardware kennt 64-Byte-Blöcke, keine C-Namen.

## Fragen? — Kohärenz

- Warum Bus Snooping?
- Exclusive vs. Modified?
- Warum erzeugt Write auf Shared Bus-Verkehr?

## Notizen

Als Nächstes: Race Conditions — MESI lügt nicht, versteht aber kein `i++`.

## Software: gemeinsamer Zähler

- MESI: Caches *kohärent* (pro Adresse)
- Semantik von `zaehler++` (RMW) ist MESI egal
- Beide Kerne: `zaehler++` auf globalem `int` — Erwartung: `+2`

## Notizen

Einfachste Aufgabe — parallel gefährlich.

## Anatomie von `i++`

In C wirkt atomar — in Assembler typisch RMW:

```
lw    t0, 0(s1)    # Read
addi  t0, t0, 1     # Modify
sw    t0, 0(s1)    # Write
```

- Drei Schritte — können sich zwischen Kernen überlappen

## Notizen

CPU addiert nicht „im RAM“ — immer Register-Umweg.

## Race Condition

1. Core 0 liest 0
2. Core 1 liest 0 (Shared)
3. Core 0 schreibt 1
4. Core 1 schreibt ebenfalls 1 (seine lokale Rechnung)

Ergebnis oft 1 statt 2 — ein Update verloren.

## Notizen

Wettlauf: beide rechnen auf demselben Startwert.

## Naive Software-Locks

```
/* ohne volatile: Compiler darf den Spin-Loop wegoptimieren */
while (lock == 1) { }
lock = 1;
zaehler++;
lock = 0;
```

Idee: Mutual Exclusion — aber Check und Set sind getrennt.

## Notizen

Schloss-Metapher stimmt logisch — atomar nicht. `volatile` verhindert Compiler-Optimierung des Spins, macht Check+Set aber nicht atomar.

## Folie

29 / 51

### Warum Software-Locks scheitern

- `lock == 0` lesen und `lock = 1` setzen: zwei Ops
- Beide Kerne sehen „frei“ → beide betreten den kritischen Abschnitt
- Race auf dem Lock selbst — braucht Hardware-Atomics

## Notizen

Problem nur verschoben — Silizium muss RMW unteilbar machen.

## Folie

30 / 51

### RISC-V A-Extension

- Atomic Instructions: unteilbares RMW für eine Adresse zwischen Harts (gemäß Spezifikation)
- Grundlage für Multicore-OS und sichere Shared-Zähler
- Zwei Familien: LR/SC und AMO
- Hinweis: paralleles DMA auf dieselbe Adresse bleibt Software-Risiko

## Notizen

Ohne Atomics kein korrektes Multicore-OS auf Shared Memory. DMA braucht eigene Sync-Disziplin.

## Folie

31 / 51

### LR/SC

- `lr.w`: lesen + Reservation/Monitor auf Adresse
- `sc.w`: schreiben nur, wenn Reservation noch gültig
- Sonst: Store fehlgeschlagen → Software wiederholt

## Notizen

Überwachtungszaun um die Adresse zwischen Load und Store.

**LR/SC: Erfolg**

- Core 0: `lr.w` → frei → `sc.w` erfolgreich
- Lock exklusiv übernommen

**Notizen**

Erfolgspfad: wie RMW, Monitor still.

**LR/SC: Fehlschlag**

- Beide: `lr.w` nahezu gleichzeitig
- Core 0: `sc.w` ok
- Core 1: `sc.w` scheitert (Reservation gebrochen / Kohärenz)
- Software von Core 1 wiederholt die Schleife

**Notizen**

Zweiter Kern prallt ab — Zustand bleibt konsistent.

**LR/SC: Spinlock in Assembler**

```

acquire:
    lr.w    t0, (a0)          /* Lock-Adresse in a0 */
    bnez    t0, acquire       /* belegt? erneut versuchen */
    li      t1, 1
    sc.w    t2, t1, (a0)      /* t2=0 bei Erfolg */
    bnez    t2, acquire       /* Reservation gebrochen → Retry */
    /* kritischer Abschnitt */
release:
    sw      zero, 0(a0)       /* Lock freigeben */
  
```

- `sc.w` liefert Status in `t2`: 0 = Erfolg, sonst Fehlschlag
- Typisches Muster für Mutex/Spinlock auf RISC-V ohne AMO

## Notizen

Genau diese Retry-Schleife erwartet die Klausur: LR, Test, SC, bei Fehler zurück. Release oft als einfacher Store (ggf. mit fence).

## Folie

35/51

### AMO-Befehle

- z. B. `amoadd.w`: Add am Speicherpfad
- RMW in Hardware „hinter der Tür“
- Adresse kurz für andere Zugriffe geschützt

## Notizen

Ein Befehl statt explizitem LR/SC-Loop für einfache Ops.

## Folie

36/51

### C11: `<stdatomic.h>`

```
#include <stdatomic.h>

_Atomic int zaehler = 0;

void isr_oder_core1(void) {
    atomic_fetch_add(&zaehler, 1);
}
```

- Compiler erzeugt LR/SC oder AMO — kein Inline-Asm nötig

## Notizen

Modernes C trifft RISC-V-A — Brücke Software/Hardware.

## Deadlocks

- Locks gegen Races — Risiko: zirkuläres Warten
- Core 0 hält A, will B; Core 1 hält B, will A
- Hardware rettet nicht — Reihenfolge der Lock-Hierarchie in Software

## Notizen

Atomics schützen Daten, nicht vor schlechter Lock-Ordnung.

## Fragen? — Race Conditions

- Race bei `zaehler++`?
- Warum reicht normales `int lock` nicht?
- Wie sichert LR/SC den Zugriff?

## Notizen

Kohärenz = Wahrheit; Atomics = unteilbare Updates. Als Nächstes: Amdahl und Semesterfazit.

## Amdahls Gesetz

- Nicht alles ist parallelisierbar: serieller Anteil  $s$ , paralleler Anteil  $p = 1 - s$
- Speedup bei  $N$  Kernen:

$$S(N) = \frac{1}{s + \frac{p}{N}}$$

- Grenze für  $N \rightarrow \infty$ :  $S_{\max} = 1/s$

## Notizen

Mehr Kerne helfen nur dem parallelen Anteil. 10 % seriell → maximal 10× Speedup — egal wie viele Kerne.

### Amdahl: Zahlenbeispiel

- Programm: 80 % parallel, 20 % seriell ( $s = 0,2$ )
- 4 Kerne:  $S(4) = 1/(0,2 + 0,8/4) = 1/0,4 = 2,5$
- 100 Kerne:  $S(100) \approx 1/(0,2 + 0,008) \approx 4,8$
- Trotz 100 Kernen weit unter  $5\times$  — der serielle Rest dominiert

### Notizen

Klausurtypisch: Einsetzen und interpretieren. Optimierung des seriellen Pfads lohnt oft mehr als „noch ein Kern“.

### Unser fertiges SoC (Rückblick)

Dual-Core-Idee mit Bausteinen aus MCT3:

- Kerne: Pipeline/OoO-Konzepte, Atomics
- Speicher: L1 (MESI), Hierarchie, Execute-in-Place aus Flash
- Zeit/IRQ: CLINT, PLIC, Timer/PWM, WDT
- Busse: UART, SPI, I2C
- Offload: DMA; optional Edge-AI-Pfad

### Notizen

Vom Addierer zum überblickbaren Microcontroller-SoC.

### Drei Flaschenhälse — Lösungen

| Feind                   | Lösungspfad                       |
|-------------------------|-----------------------------------|
| Memory Wall             | Caches, DLP (Vektoren/NPUs)       |
| Control Hazards         | Prediction, Unrolling, Scheduling |
| CPU blockiert durch I/O | Interrupts, DMA                   |

### Notizen

Optimierung heißt: welchen der drei Feinde treffe ich?

## C-Code vs. Hardware

Ohne Hardwareblick typisch:

- Cache-feindliche Schleifen (Layout)
- Busy-Waiting auf UART/Status
- Shared `i++` ohne `_Atomic` / falsches `volatile`

Mit Systemblick: Caches, Pipeline, DMA und Kohärenz mitdenken.

### Notizen

Röntgenblick: eine C-Zeile kann harmlos oder systemtötend sein.

## Infos zur Klausur

- Schriftlich, Dauer laut Modulbeschreibung / Ankündigung
- Hilfsmittel: nicht programmierbarer Taschenrechner; keine Skripte
- Fokus: Konzepte, kleine Rechnungen (z. B. AMAT), Skizzen, C-Ausschnitte

### Notizen

Problemlösung vor Auswendiglernen von Datenblatt-Adressen.

## Klausurfokus: Speicher

- AMAT bei Hit-Rate und Miss-Penalty
- Direct-Mapped vs. satzassoziativ; Thrashing
- Cache-Line: Tag, Index, Offset, Valid, Dirty
- Write-Back + DMA; MESI bei Multicore

### Notizen

Adresse zerlegen und Hierarchiekosten rechnen können.

**Klausurfokus: Hardwarenahes C**

- Row- vs. Column-Major / Lokalität
- MMIO-Pointer
- `volatile` bei MMIO/IRQ (Compiler darf Zugriffe nicht wegoptimieren) — ersetzt keine `Atomics`
- Races erkennen; `_Atomic` / `stdatomic` als Lösung

**Notizen**

„Warum ist dieser Schnipsel gefährlich?“

**Klausurfokus: System**

- Polling vs. IRQ; CLINT vs. PLIC
- Timer/Prescaler, PWM Duty Cycle; WDT
- UART/SPI/I2C-Eigenschaften
- DMA: SRC, DST, Increment
- OoO: Renaming, ROB (Grundidee)

**Notizen**

Display: SPI oder I2C — und warum?

**Was nicht im Fokus steht**

- Keine auswendig gelernten Chip-Basisadressen (ggf. gegeben)
- Keine langen Assembler-Programme (MCT1)
- Keine Halbleiterphysik/Leckstrom-Formeln

**Notizen**

Konzepte und Prinziprechnungen — keine Datenblatt-Trivia.

### Offene Fragerunde

- Welche Bausteine des SoCs sind noch unklar?
- Was soll wiederholt werden?

### Notizen

Nutzen Sie die Gelegenheit vor der Klausurphase.

### Zusammenfassung Block 9

- SMP: private L1, gemeinsamer Speicher → Kohärenz nötig
- MESI: M/E/S/I; Write auf Shared → Invalidate
- Race: `i++` ist RMW — MESI reicht nicht; Atomics (LR/SC, AMO)
- Amdahl: serieller Anteil begrenzt den Multicore-Speedup
- Klausur: AMAT, Mapping, IRQ/Timer, PWM, DMA, OoO-Idee, MESI

### Notizen

Kohärenz hält Caches ehrlich; Atomics halten Updates unteilbar; Amdahl hält Erwartungen realistisch.

### Abschluss und Ausblick

- Theorie MCT3 abgeschlossen
- Labor-Finale: Probeaufgaben, Renode, offene Fragen

Vielen Dank für Ihre Mitarbeit im Semester!

### Notizen

Freude, Sie durch die SoC-Ebenen begleitet zu haben — viel Erfolg in der Klausur.